



PROJET NFE204

Etude de Couchbase : création d'une base de données, interrogation, exploitation et recherche au sein du système

Résumé

Mise en application des principes vus dans le cadre de l'unité d'étude NFE204, dirigée par Philippe Rigaux, sur un environnement Couchbase avec des données issues de la base Socrata Open Data de la ville de New-York.

Sylvain CAZABAN-MAZEROLLES

Auditeur NFE204 Semestre 2 - 2016

Table des matières

I. Introduction.....	1
II. Couchbase	2
1. Présentation de Couchbase	2
2. Installation de Couchbase.....	2
3. Architecture et description du système Couchbase	3
Architecture SNA.....	3
Le gestionnaire de données	3
la gestion du cache des métadonnées	4
Le gestionnaire de cluster	4
Le moteur de requête	4
La réplication.....	4
4. Premiers tests d'interrogation.....	5
III. Données JSON et import dans Couchbase	5
1. La source de données	5
2. Description du document	6
3. Création du jeu de données	7
Export initial	7
Automatisation des Exports et intégration	9
4. Insertion des données JSON	10
Créer des documents par l'API.....	10
Insérer des documents par les services REST.....	10
Insérer des documents par l'outil d'import	10
5. Requêtes sur la base.....	11
IV. Exploitation des données	12
1. Requêtes simples.....	12
2. Exploiter des données dans une feuille de calcul	12
3. Exploiter les données sous forme de carte GPS	13
4. Créer des vues sur les données	14
Reduce personnalisé et Rereduce	15
Première implémentation : un simple count(*).....	15
Deuxième implémentation : obtenir le nombre d'accidents mortels par quartiers	15
Vérification des résultats en utilisant le langage N1QL.....	16
Etude et discussion des résultats	17
V. Indexation de Couchbase par Elasticsearch	18
1. Installation et configuration du serveur Elasticsearch	19
Fichier de configuration	19

Création de l'interface entre Couchbase et Elasticsearch	20
Mise en route de la réplication du cluster Couchbase	20
Requêtes de recherche par l'interface	22
Etude et discussion des résultats	23
2. Configuration du mapping Elasticsearch	25
Job Search	25
Configuration du mapping	26
Création d'un analyseur	28
Tokenization	28
Stemming	29
Mise en place de l'analyseur pour les données issues de Couchbase	30
VI. Conclusion	32
VII. Annexes	33
1. Références	33
Concepts Couchbase	33
SOCRATA	33
CartoDB	33
Elasticsearch	33
2. Compléments	33
Requêtes N1QL avec Curl	33
3. Interfaces	34
Interface de création d'une vue dans Couchbase	34
Implémentation du MapReduce	34
4. Elasticsearch	35
Mapping	35
Analyseur	35

I. Introduction

Auditeur au CNAM en vue de l'obtention du diplôme d'ingénieur en informatique, spécialité systèmes d'information (ISI), j'ai choisi de suivre l'option bases de données avancées. J'ai auparavant déjà obtenu le titre de Concepteur-Architecte au sein du CNAM. Je ne prépare pas la certification Analyse de données massives. Cette unité d'étude fait partie de l'option choisie pour la préparation de mon diplôme d'ingénieur. Je suis actuellement Chef de projet et analyste-développeur au sein d'une entreprise dont le métier est la construction, le bâtiment et les travaux publics. Responsable de l'intranet du groupe basé sur un socle Microsoft SharePoint, je suis confronté régulièrement aux problématiques liées à la notion de gestion électronique des documents. SharePoint fonctionne sur un modèle relationnel, mais dispose de web services basés sur SOAP et REST. Les notions abordées dans le cadre de cet enseignement ne me sont pas toutes inconnues et elles complètent ma vision de la problématique de stockage d'informations mais cette fois à grande échelle, sur de grands volumes. Il est peu probable que les concepts abordés soient mis en application au sein de mon travail actuel, les bases de données documentaires distribuées étant une réponse à des problèmes très précis, mais les concepts établis pour répondre à ces enjeux sont particulièrement intéressants et permettent de mieux comprendre les systèmes basés sur ces architectures.

Parmi les systèmes NoSQL présentés par le site nosql-database.org, j'ai choisi Couchbase. Mon choix s'est effectué par le fait que ce système est très bien documenté, semble très complet et offre donc de nombreuses pistes à explorer dans le cadre d'un petit projet.

Les données étudiées dans le cadre du projet proviennent de la base de données Open Data fournie par la ville de New-York. Les documents mis à disposition sont variés, ceux choisis ici concernent les collisions des véhicules à moteur, relevés par le département de la police de la ville de New-York. J'ai choisi ces documents du fait qu'ils portent un nombre suffisant de données pour créer une base conséquente, la base est maintenue et le volume associé est de plus de 700 000 documents disponibles.

Dans un premier temps, je vais aborder le système Couchbase en passant par son installation sur un système Windows, puis par la description de son architecture.

J'exposerai ensuite les données JSON qui seront importées dans la base de données, après avoir passé en revue les façons d'envoyer les données dans Couchbase, je détaillerai le procédé construit pour récupérer et insérer les données, puis j'effectuerai les premiers tests d'interrogation.

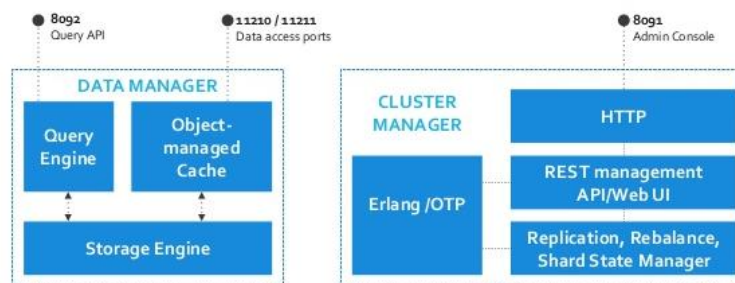
Dans une troisième partie, je montrerai quelques possibilités pour exploiter les données présentes dans le système, en passant par de premières requêtes simples. Puis par l'extraction de données, transformées en csv, je montrerai comment faire des rapports ou leur modélisation au sein d'une carte géographique. Enfin je montrerai comment créer des vues dans Couchbase, j'implémenterai les principes du mapReduce pour faire écho au cours et je discuterai des résultats obtenus comparativement au mécanisme de requête N1QL propre à Couchbase.

Dans une dernière section, je détaillerai l'installation et la configuration d'un serveur d'indexation dédié pour les données issues de Couchbase. J'établirai le mapping des données sur Elasticsearch et décrirai la mise en place d'un analyseur personnalisé.

II. Couchbase

1. Présentation de Couchbase

Couchbase est un projet open source, de base de données documentaire et distribuée. Couchbase est une solution robuste et avancée, qui gère les accès concurrent sur les documents, le passage à l'échelle, la réplication, l'équilibrage de charge, la reprise sur panne, les sauvegardes. Les documents insérés dans Couchbase sont stockés au format JSON ou comme binaires. Couchbase est un produit relativement récent qui a vu le jour en juin 2010 sous le nom de Membase. En 2011 l'équipe de Membase a annoncé sa fusion avec celle de CouchOne pour donner naissance à Couchbase en 2012. Le langage créé par Couchbase pour interroger ses bases dans un langage inspiré de SQL est appelé N1QL, prononcé « nickel ». L'architecture d'un serveur Couchbase est composée de deux grandes parties, la première gère l'accès et le stockage des données, tandis que la deuxième gère l'administration du cluster Couchbase.



Architecture d'un serveur Couchbase

2. Installation de Couchbase

J'ai choisi d'installer Couchbase sur un système Windows. Etant développeur sur le Framework .Net, Windows est le système d'exploitation que j'ai le plus couramment utilisé au travail. Je me suis donc tourné naturellement vers ce système d'exploitation. Pour installer Couchbase sur Windows, j'ai récupéré la dernière version stable, la version 4.0 en 64 bits. L'installation se fait par un outil de configuration en quelques clics. A la fin de l'installation nous accédons à la console de Couchbase par l'intermédiaire d'un navigateur web. La configuration initiale du serveur est alors possible.

Configuration du serveur :

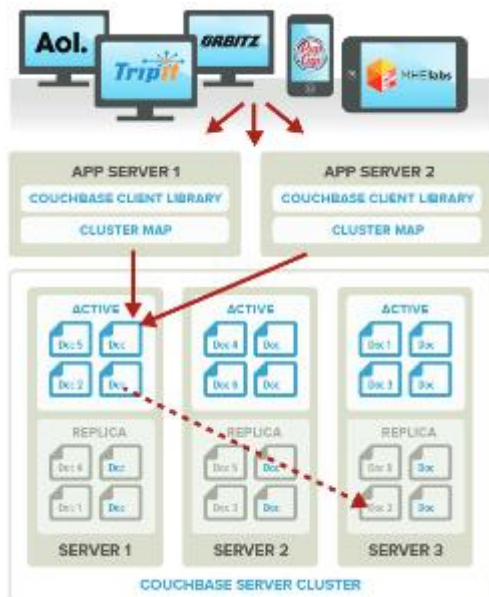
- Chemins vers les bases de données et vers les index (penser aux I/O et à l'architecture des grappes de disques).
- Hostname de l'application (permet d'accéder au serveur via un nom plutôt qu'une adresse IP si configuré dans le DNS).
- Possibilité de rejoindre un cluster déjà existant.
- Choisir les services activés sur le serveur parmi Data, Index et Query puis d'allouer l'espace mémoire RAM pour chacun de ces services. Des services peuvent être dédiés à des serveurs pour équilibrer la charge globale du cluster au sein d'une architecture n-tiers.
- Définir le mode de stockage du bucket par défaut (RAM ou Disque).
- La quantité de mémoire alloué au bucket.
- la gestion du cache des métadonnées.

- Le nombre de répliques pour la sauvegarde des données.
- Possibilité de donner une forte priorité aux accès disques aux tâches Couchbase.
- Paramètres d'authentification (nécessaires pour la mise en place du cluster).

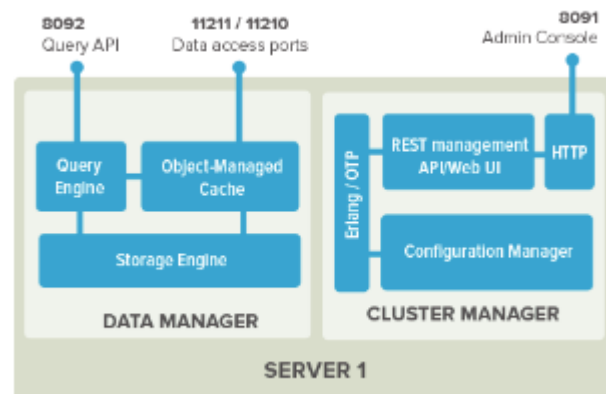
3. Architecture et description du système Couchbase

ARCHITECTURE SNA

Couchbase est fondé sur une architecture SNA pour Shared-Nothing Architecture. Il consiste en une agrégation de serveurs derrière un ou plusieurs serveurs applicatifs frontaux.



Architecture du cluster



Architecture d'un serveur

Les documents sont distribués uniformément au travers du cluster et stockés dans des conteneurs de documents appelés buckets. Les buckets sont des groupements logiques de ressources physiques au sein d'un cluster. Quand un bucket est créé, de la mémoire RAM lui est alloué pour mettre ses données en cache, et son nombre de répliques est défini. Chaque bucket est découpé en 1024 partitions logiques appelés vBuckets. Ces partitions sont mappées à des serveurs du cluster, les données du mapping sont stockées dans le cluster map.

Chaque document ajouté à la base de données se voit attribuer un ID, qui est hashé en CRC32, puis il est distribué sur les serveurs de destination. Il est d'abord répliqué, puis il est mis en mémoire, puis stockés de façon asynchrone sur disque.

Chaque bucket est répliquable 3 fois dans un cluster. Une seule des partitions est active à la fois. En cas de coupure d'un serveur, l'une des deux autres répliques inactive est désignée comme active.

LE GESTIONNAIRE DE DONNEES

D'un point de vu logique, les serveurs Couchbase sont constitués d'un gestionnaire de données, lui-même composé de trois éléments : le moteur de requêtes, le cache des objets gérés, et le moteur de stockage.

L'Object-Managed Cache stocke en mémoire les tables de hashage représentant les documents et leurs métadonnées. Le contenu lui-même des documents peut être ajouté aux métadonnées. Cela

permet au serveur de garantir des latences très faibles pour servir les requêtes. Les requêtes sont journalisées dans un fichier, ce qui permet au serveur d'effectuer un warmup à partir de ce fichier en cas de redémarrage.

Le Storage Engine de Couchbase ne fonctionne que par ajout, il n'y a jamais de mise à jour directe. Les ajouts se font toujours à la fin des fichiers de façon séquentielle. Ce qui garantit des temps d'accès très rapide. Le moteur utilise des fichiers multiples pour le stockage et gère les accès aux documents par des pointeurs. Ces fichiers sont soit des fichiers de données, soit des vues, soit des index. Qui sont eux-mêmes actifs ou passifs répliqués. Les fichiers de données sont organisés en structure d'arbres B, un principe de stockage équilibré de données favorisant la recherche, l'insertion et la suppression.

LA GESTION DU CACHE DES METADONNEES

Il est possible de définir la façon dont la mémoire des serveurs se vide. Soit en vidant uniquement les valeurs propres aux données et en conservant les métadonnées et les clés, soit en supprimant tout. Conserver les clés et les métadonnées donnera de meilleures performances à la base grâce à la persistance en mémoire de données précédemment utilisées, donc potentiellement réutilisables. Le choix d'éjecter toutes les données peut être fait si l'espace mémoire est faible sur le serveur, au détriment des performances.

LE GESTIONNAIRE DE CLUSTER

Le gestionnaire de cluster supervise la configuration des serveurs et leurs interactions au sein du cluster. Il gère la réplication et l'équilibrage de charge. Lorsqu'un serveur n'est plus joignable, le serveur orchestrateur de cluster notifie les autres machines et promeut les données répliquées inactives qui étaient actives au sein du serveur injoignable. La carte du cluster est mise à jour sur tous les nœuds du cluster. C'est de cette façon que Couchbase gère la reprise sur panne.

LE MOTEUR DE REQUETE

Les index primaires sont gérés par Couchbase. Les index secondaires sont définis lors de la création de documents de conception et des vues associées. Chaque documents de conception est propre à un bucket et porte en lui les vues qui lui sont associées. Au sein de chaque vues sont définies les fonctions de map et de reduce. La fonction de map définit les attributs sur lesquels seront établis les index secondaires. Les index sont eux aussi distribués, chaque serveur fait son propre indexage.

Chaque requête reçue par le serveur applicatif est distribuée à tous les serveurs du cluster, les résultats sont agrégés avant d'être renvoyés. La construction d'une vue est incrémentale, chaque fois qu'une vue est accédée, elle est appliquée à tous les documents nouvellement insérés ou modifiés.

LA REPLICATION

Appelé XCDR pour Cross Datacenter Réplication, elle permet de répliquer des données entre clusters distincts, situé dans des zones géographiques différentes par exemple. La réplication interne au cluster et par le XCDR surviennent en même temps. La réplication peut se faire au niveau du bucket, pas seulement sur le cluster en entier. La réplication supporte la réplication en continue des données, détecte les pannes de cluster pour mettre à jour à la topologie distante et est conçue pour optimiser la bande passante (déduplication). En cas de détection de conflit de mises à jours de documents sur des Datacenter distants, le document élu sera celui ayant reçu le plus de mises à jour.

4. Premiers tests d'interrogation

Couchbase vient avec un certain nombre d'utilitaires, pour les appeler simplement depuis une ligne de commande, il faut ajouter le chemin vers les binaires `C:\Program Files\Couchbase\Server\bin\` dans le PATH de Windows. Lors de l'installation de Couchbase, il est possible d'installer des jeux de données pour servir d'exemple aux tests.

L'utilitaire permettant d'effectuer des requêtes N1QL est `cbq.exe`.

Une première requête d'interrogation vers notre bucket échouera s'il n'y a pas d'index sur celui-ci.

Pour créer l'index, effectuer la commande :

```
CREATE PRIMARY INDEX ON `beer-sample` USING GSI;
```

Une requête `SELECT` peut maintenant être effectuée.

```
SELECT DISTINCT type FROM `beer-sample`;
```

La commande donnera toutes les valeurs possibles pour la propriété `type` des documents JSON du bucket `beer-sample`.

III. Données JSON et import dans Couchbase

1. La source de données

Dans le cadre du projet je suis allé à la recherche de données à exploiter. J'ai retenu le choix de document au format JSON par commodité, en effet les systèmes NoSQL les plus documentés fonctionnant sur Windows étaient tournés vers ce format de données.

J'ai recherché des sources de données selon les critères suivants :

- volume disponible (nombre d'enregistrements),
- périodicité de mise à jour,
- présence de données récentes,
- possibilité d'interroger les données par web service et/ou une API.

Après quelques recherches, j'ai découvert la base de données open data de la ville de New-York. La base de données de la ville de New-York est accessible au travers d'une application cloud appelée Socrata. Socrata est le nom de l'entreprise et du logiciel. Celui-ci est destiné aux administrations souhaitant publier des données sous forme d'open data à des utilisateurs d'internet sans que ceux-ci aient besoin d'une expertise particulière pour visualiser les données.

Le contenu mis à disposition à travers Socrata par le programme Open Data de la ville de New-York est conséquent, il est réparti dans les catégories suivantes : Business, City Government, Education, Environment, Health, Housing & Development, Public Safety, etc.

J'ai choisi d'extraire les données disponibles dans la catégorie sécurité, liées aux collisions de véhicules à moteurs dans la ville de New-York. Ces données ont retenu mon attention car elles détiennent pour chaque document suffisamment d'information pour qu'elles puissent être analysées, le contenu de la base est volumineux (plus de 700.000 documents) et les documents sont

mis à jour régulièrement, toutes les semaines. Pour un jour, ce sont environs 500 documents qui sont créés. La base est alimentée avec des données partant du 1er juillet 2012.

Socrata permet d'interroger ses données par l'intermédiaire de SODA, qui est le nom de son API, pour Socrata Open Data API. Des kits de développement pour les principaux langages de programmation sont mis à disposition.

Je n'ai pas utilisé l'API SODA pour récupérer les documents de Socrata pour la ville de New-York, l'API s'installe facilement sous Visual Studio via l'utilitaire d'installation de package NuGet, mais elle a pour but d'interroger les données directement sur cette plateforme via la formation de requêtes ressemblant à SQL, appelées SoQL, puis à analyser les données dans l'application. Ma démarche a pour but d'exporter les données brutes, pour cela je n'ai pas eu besoin de l'API.

2. Description du document

Les documents extraits de la base de données des collisions de véhicules sont constitués des attributs suivant :

```
{
  "number_of_persons_killed" : "0",      //Nombre de personnes tuées
  "vehicle_type_code1" : "TAXI",         //Code du type de véhicule
  "zip_code" : "10012",                  //Code Postal
  "vehicle_type_code2" : "TAXI",
  "location" : {                          //Coordonnées GPS de lieu de l'accident
    "needs_recoding" : false,
    "longitude" : "-73.999917",
    "latitude" : "40.7269334"
  },
  "number_of_motorist_injured" : "1",    //Nombre d'automobilistes tués
  "date" : "2014-05-17T00:00:00",        //Date de l'accident
  "off_street_name" : "LAGUARDIA PLACE",  //Rue du croisement
  "time" : "0:10",                       //Heure
  "on_street_name" : "WEST HOUSTON STREET", //Rue où se situe l'accident
  "number_of_pedestrians_injured" : "0", //Nombre de piétons blessés
  "number_of_cyclist_killed" : "0",      //Nombre de cyclists tués
  "longitude" : "-73.9999170",
  "number_of_cyclist_injured" : "0",      //Nombre de cyclists blessés
  "unique_key" : "336440",                //Identifiant unique
  "borough" : "MANHATTAN",               //arrondissement
  "number_of_pedestrians_killed" : "0",   //Nombre de piétons tués
  "contributing_factor_vehicle_1" : "Pavement Slippery",
  "number_of_motorist_killed" : "0",      //Nombre d'automobilistes tués
  "number_of_persons_injured" : "1",     //Nombre de personnes blessées
  "contributing_factor_vehicle_2" : "Unspecified",
  "latitude" : "40.7269334"
}
```

Le champ `contributing_factor_vehicle` peut exister cinq fois, noté de 1 à 5. C'est la cause de l'accident. Le champ `vehicle_type_code1` peut exister cinq fois, noté de 1 à 5. C'est le type du véhicule causant l'accident. Nous sommes en présence d'un document JSON portant la donnée sans référence externe. Les types et les codes sont des libellés portés par le document. Il n'existe pas de base donnant tous les types et codes possibles.

3. Création du jeu de données

EXPORT INITIAL

Pour récupérer les données de la base des collisions de véhicule de la ville de New-York, il est possible d'utiliser des requêtes http auprès du web service. Concernant cette base, plusieurs web services sont disponibles, délivrant des fichiers XML, JSON ou CSV.

Pour appeler le web service fournissant des résultats JSON, il faut faire un GET sur l'adresse suivante :

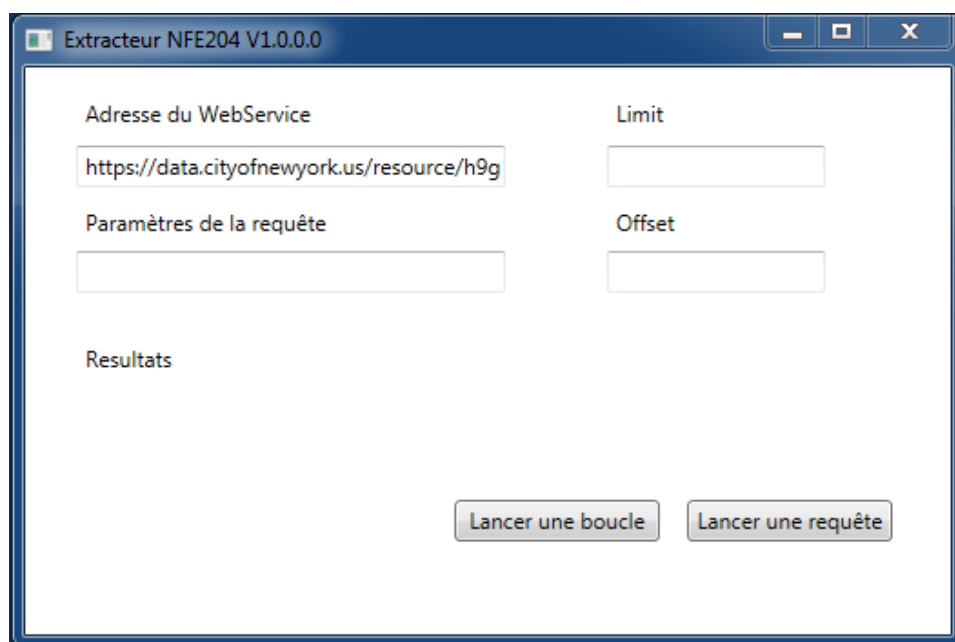
```
https://data.cityofnewyork.us/resource/h9gi-nx95.json
```

Quelques mots SoQL, des paramètres utilisés dans l'url http, permettent de filtrer les résultats lors de l'appel au web service. Il est également possible de passer les mots SoQL dans un seul paramètre \$query.

Voici la requête qui m'a permis d'obtenir le nombre de documents dans la base :

```
https://data.cityofnewyork.us/resource/h9gi-nx95.json?$query= SELECT  
Count(unique_key)
```

Pour extraire les données j'ai conçu une application Windows WPF.



Application permettant d'appeler le web service pour extraire les données

L'application que j'ai développé est relativement simple et mériterait d'être améliorée avec des tests plus poussés. Il reste à l'état de prototype, ses fonctionnalités suffisant pour récupérer les données. Le programme prend en paramètre l'adresse du web service et les paramètres de la requête pour lancer une requête simple. Pour lancer des requêtes en boucle, il faut saisir le nombre d'enregistrement par boucle (paramètre limit) et le point de départ (paramètre offset).

Le programme fait un appel à l'adresse http construite en fonction des paramètres. Le résultat JSON est inséré dans un lecteur de flux (StreamReader) puis sauvegardé dans un fichier avec l'extension JSON.

En cas de boucle de requête, le programme s'arrête quand il n'y pas de résultats dans la réponse http. Dans ce cas le résultat est un tableau vide« [] ».

Pour extraire les 700.000 enregistrements, j'ai effectué une boucle de requêtes en partant de 0 par pas de 10.000, en triant les résultats par clef unique ascendante. Le pas choisi permettait que le service web réponde dans les temps, sinon un timeout survenait, et d'avoir un export initial ne contenant pas trop de fichiers.

Il a fallu plusieurs heures pour récupérer tous les documents, les réponses http étant assez lentes depuis les Etats-Unis. L'export est constitué de 77 fichiers de 10.000 documents JSON. Le programme a nommé les fichiers en fonction du pas et du nombre d'éléments par pas, pour un format de ce type : data-[nième élément]-[nième élément + pas].json

File Name	File Type	Modified	Size
data-0-10000.json	Fichier JSON	10/03/2016 15:55:14	8,619,913
data-10000-20000.json	Fichier JSON	10/03/2016 15:55:32	8,693,472
data-100000-110000.json	Fichier JSON	10/03/2016 14:41:28	8,376,595
data-110000-120000.json	Fichier JSON	10/03/2016 14:41:50	8,642,771
data-120000-130000.json	Fichier JSON	10/03/2016 14:42:12	8,639,703
data-130000-140000.json	Fichier JSON	10/03/2016 14:42:36	8,772,846
data-140000-150000.json	Fichier JSON	10/03/2016 14:43:00	8,628,462
data-150000-160000.json	Fichier JSON	10/03/2016 14:43:24	8,706,229
data-160000-170000.json	Fichier JSON	10/03/2016 14:43:48	8,652,555

Extraction des données au format JSON sous forme de fichiers de 10 000 documents

L'import de documents dans Couchbase ne peut se faire que de façon atomique, un fichier ne doit contenir qu'un seul document, contrairement à MongoDB par exemple où il est possible d'importer un tableau de documents.

J'ai donc du écrire un script python qui a extrait pour chacun de ces fichiers, chaque ligne de données JSON en se basant sur le champ unique_key et créé un nouveau document JSON nommé par sa clé.

```
7 path = './rawdata/'
8
9 for filename in os.listdir(path):
10
11     data = json.load(open(str(path+filename)))
12
13     for i in range(0, len(data)):
14         id = data[i]['unique_key']
15         outfile = open('./output/' + str(id) + '.json', 'w')
16         json.dump(data[i], outfile)
17         outfile.close()
18
```

Script Python pour générer 700 000 fichiers

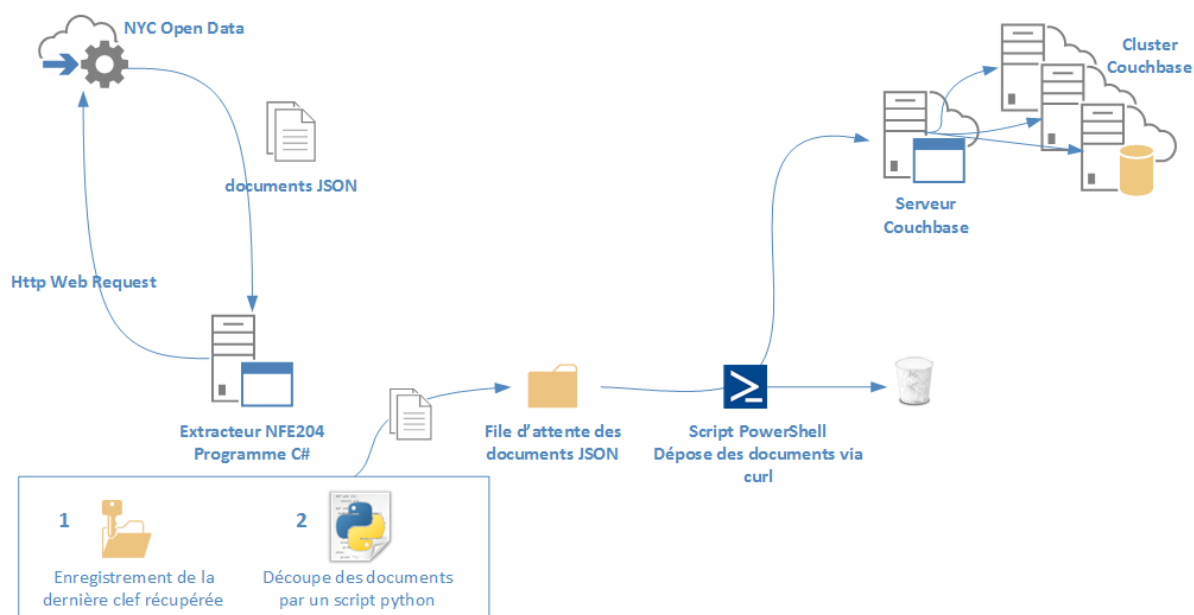
A l'issu du traitement qui a duré plusieurs dizaines de minutes, plus de 700 000 documents de 1 ko ont été créés.

3400834.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400835.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400836.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400837.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400851.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400869.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400870.json	12/03/2016 12:04	Fichier JSON	1 Ko
763 968 élément(s)			

Résultat du script 763 968 fichiers créés.

AUTOMATISATION DES EXPORTS ET INTEGRATION

Il n'existe pas de critères permettant de récupérer les nouveaux documents JSON disponibles sur la base de données depuis une certaine date. En effet les documents apparaissent par intervalles, sans tri dans les dates. Le séquençement des clefs uniques n'est pas lié à la date de l'accident. Néanmoins, les numéros de clés uniques se suivent même s'il existe parfois des sauts de chiffre. Ce qui laisse place à un schéma d'exportation.



Processus d'exportation des données JSON et importation dans Couchbase

L'automatisation va pouvoir être établie selon le processus suivant :

1. Une tâche planifiée exécute le programme WPF, celui-ci effectue un appel au web service avec pour paramètre une sélection des documents dont la clef unique est supérieure à la dernière récupérée. Exemple de paramètre : `$where unique_key > lastuniquekey &$limit=1000 &$offset=X`. Les fichiers JSON récupérés sont stockés dans un répertoire temporaire. La clef la plus haute des documents récupérés est stockée pour les prochains appels.

2. une tâche planifiée exécute un script python qui prend chaque document JSON et le découpe en fichiers contenant un seul document JSON et les dépose dans un répertoire constituant la file d'attente.
3. Une tâche planifiée exécute un script PowerShell qui lance la commande curl vers le cluster Couchbase pour déposer chaque fichier contenu dans le répertoire de file d'attente. Chaque fichier déposé est ensuite supprimé.

4. Insertion des données JSON

Nous allons voir dans cette partie comment importer les documents JSON dans Couchbase.

CREER DES DOCUMENTS PAR L'API

Couchbase permet d'insérer des documents par l'intermédiaire de son API Java. Un exemple :

```
JsonObject content = JsonObject.empty().put("name", "Michael");
JsonDocument doc = JsonDocument.create("docId", content);
JsonDocument inserted = bucket.insert(doc);
```

D'autres environnements sont supportés : .Net, PHP, Node.js, Python, Ruby.

INSERER DES DOCUMENTS PAR LES SERVICES REST

Il est également possible de déposer des documents en utilisant REST. Il faut pour cela mettre en place un serveur dédié en tant que portail de synchronisation, le « Sync Gateway ». En effet, il n'est pas possible de déposer des documents directement en effectuant une requête http via curl vers le cluster Couchbase. Néanmoins les services REST sont bien proposés, mais pour effectuer des opérations de gestion sur le cluster, les buckets, les vues.

INSERER DES DOCUMENTS PAR L'OUTIL D'IMPORT

Dans le cadre du projet, je vais insérer les données récupérées initialement en effectuant un import en masse. Ce moyen est généralement utilisé après une installation initiale de Couchbase, ce qui est le cas ici. Pour insérer un grand volume de données en une seule fois, voici comment on procède. Premièrement, créer le bucket qui va accueillir les données. Il est possible de créer un bucket par l'interface web ou par ligne de commande. Voici la démarche par ligne de commande, en utilisant l'utilitaire couchbase-cli.

```
couchbase-cli bucket-create -c localhost:8091 -u admin -p password --
bucket=NYC --bucket-type=couchbase --bucket-ramsize=1000
```

La commande va créer un bucket nommé NYC, de type Couchbase (stocké sur disque) avec un quota de mémoire RAM alloué de 1 GB.

Couchbase Buckets

Create New Data Bucket

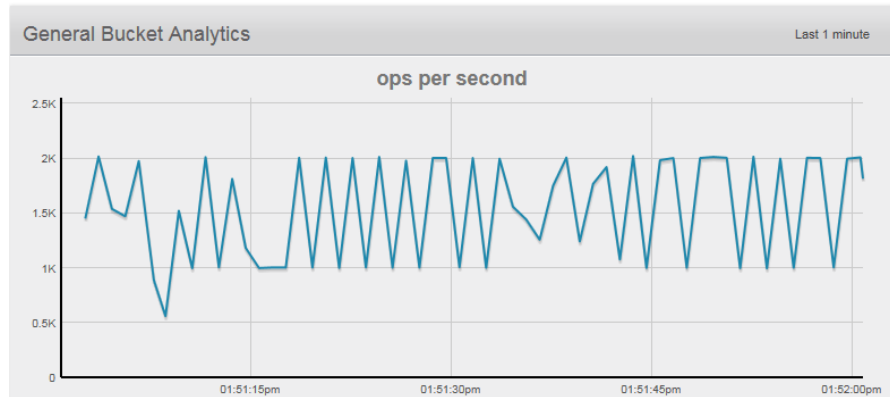
Bucket Name	Data Nodes	Item Count	Ops/sec	Disk Fetches/sec	RAM/Quota Usage	Data/Disk Usage	
▼ NYC	1	0	0	0	34MB / 1000MB	315KB / 34B	DocumentsViews
Access Control: Authentication		Replicas: 1 replica copy		Compaction: Not active		CompactEdit	
Cache Metadata: Value Ejection		Disk I/O priority: Low					

Bucket vide NYC créé par la ligne de commande

Nous importons ensuite les données. L'import des données peut se faire depuis un fichier ZIP ou depuis un répertoire. La ligne de commande s'utilise par le biais de l'utilitaire cbdocloader.

```
cbdocloader -n localhost:8091 -u admin -p password -b NYC  
E:\NFE204DATA\Bulk\output
```

Les fichiers sont importés, la console web indique 2000 opérations par seconde.



IOPS sur le bucket NYC lors de l'import

Quelques minutes plus tard, les 763968 fichiers d'import initial sont présents dans le bucket.

Couchbase Buckets							Create New Data Bucket
Bucket Name	Data Nodes	Item Count	Ops/sec	Disk Fetches/sec	RAM/Quota Usage	Data/Disk Usage	
NYC	1	763968	0	0	675MB / 1000MB	377MB / 495MB	Documents Views

Bucket NYC contenant 763968 documents JSON

5. Requêtes sur la base

Notre premier import effectué, notre base est prête à l'emploi et nous pouvons commencer l'utiliser. Avant tout, commençons par créer notre index, qui est obligatoire pour effectuer nos requêtes. On lance l'utilitaire de requête en tapant cbq en ligne de commande.

```
CREATE PRIMARY INDEX ON NYC USING GSI;
```

La commande `SELECT COUNT(*) FROM NYC;` retourne 763968, ce qui démontre la bonne exécution de la requête.

Il est également possible de passer des requêtes http en utilisant curl.exe.

```
curl -v http://localhost:8093/query/service -d "statement=SELECT COUNT(*)  
FROM NYC;"
```

IV. Exploitation des données

1. Requêtes simples

Pour effectuer des requêtes vers la base de données, il est assez commode d'utiliser dans un premier temps le langage propre à Couchbase, N1QL, en utilisant l'outil de requête cbq.exe. Par exemple, pour obtenir la liste des principaux facteurs d'accident, on saisira la requête suivante :

```
SELECT COUNT(unique_key) AS Num, contributing_factor_vehicle_1 FROM NYC
GROUP BY contributing_factor_vehicle_1 ORDER BY Num DESC;
```

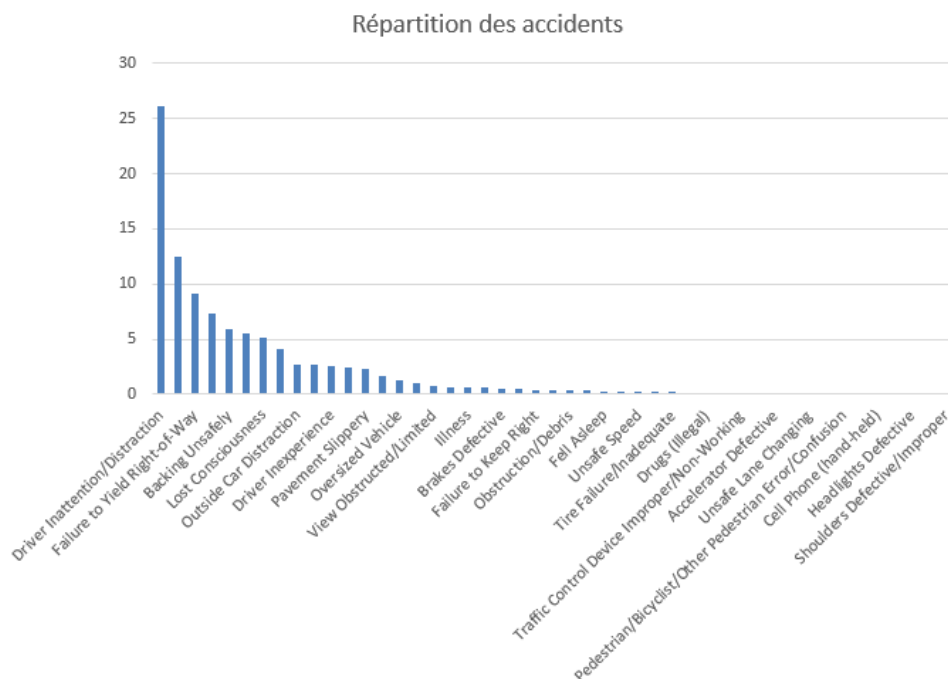
Le résultat s'affiche dans la fenêtre de l'invite de commande mais ne peut pas être exporté.

2. Exploiter des données dans une feuille de calcul

En utilisant curl, nous pouvons former des requêtes http pour récupérer nos données. Il sera alors possible d'ajouter une sortie vers un fichier texte.

```
curl -v http://localhost:8093/query/service -d "statement=SELECT
COUNT(unique_key) AS Num, contributing_factor_vehicle_1 FROM NYC GROUP BY
contributing_factor_vehicle_1 ORDER BY Num DESC;" > E:\output.txt
```

La requête curl passe en paramètre une requête SQL mais nous pouvons aussi utiliser les vues. Nous le verrons plus loin. A l'aide d'un convertisseur de format JSON vers CSV, tel que le web service précisé en annexe, il est possible d'exploiter les résultats dans un tableur. J'ai choisi d'utiliser un service web, la fonctionnalité de conversion étant simple et disponible. Dans le cadre d'une industrialisation du processus, nous préférierions établir notre propre programme de conversion. Une fois le document csv obtenu, nous pouvons modéliser nos résultats sous forme d'un graphe.



Répartition des causes d'accidents tous critères confondus, en pourcentage.

Une première analyse montre que dans la moitié des cas, la cause de l'accident n'est pas précisée, j'ai donc retiré les données liées. Rapidement nous voyons que le premier facteur d'accident est l'inattention au volant pour un cas sur quatre. Suivent majoritairement, le non-respect du cédez le passage, la marche arrière sans contrôle, la perte de conscience, la distraction perçue en dehors du véhicule, l'inexpérience, la chaussée glissante, le véhicule trop large, la vision obstruée.

3. Exploiter les données sous forme de carte GPS

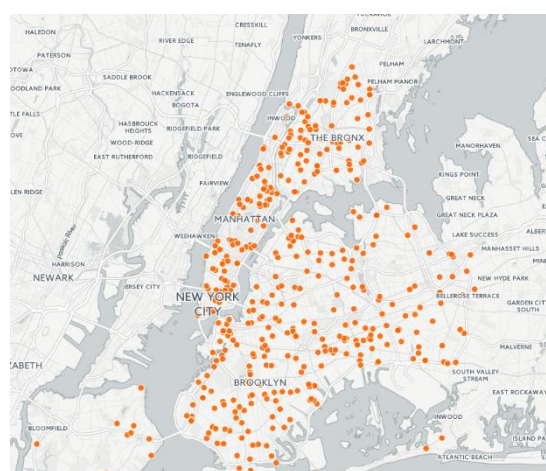
Les données relatives aux collisions détiennent les coordonnées GPS des lieux de l'accident. Il devient donc possible de représenter les lieux des accidents sur une carte. Pour appliquer le principe, je suis allé à la recherche d'un service permettant de de l'appliquer. Je me suis servi de l'application CartoDB, qui permet de représenter sur une carte des localisations en alimentant un jeu de données contenant des coordonnées GPS. L'application est gratuite, mais pour un nombre limité d'enregistrements dans un jeu de données.

Pour mettre en pratique le principe, j'ai donc effectué des requêtes sur Couchbase pour récupérer les données suivantes :

- 93.000 premiers enregistrements de la base de données, sans critère de sélection. Carte 1.
- Les collisions au 31 décembre 2015. Carte 2.
- Les collisions dans Brooklyn, en janvier 2016. Carte 3.



Carte 1 - 93000 collisions



Carte 2 – le 31 décembre 2015



Carte 3 – Brooklyn Janvier 2016

Avec ce type de modélisation, il devient possible de mettre en évidence un large éventail de causalité dans les collisions. Par exemple, en obtenant les données météorologiques de la ville de New-York sur les dernières années, par croisement des données issues de la base de collisions, il serait possible de montrer que certaines zones sont plus accidentogènes que d'autres en cas d'intempéries. De même, si les données relatives au trafic routier étaient récupérées, il serait possible d'analyser l'impact de la densité de trafic sur le nombre d'accidents et leur localisation.

CartoDB est un outil puissant, qui permet d'être exploité directement au sein d'un large type de serveurs web. Il permet également d'interroger des données par web service situé sur d'autres serveurs. Ses possibilités sont nombreuses et il est un exemple de mise en application des modèles de données accessibles en Open Data.

4. Créer des vues sur les données

Les vues dans Couchbase permettent à partir d'un bucket de créer un index sur la donnée en affichant les champs et informations souhaitées. Les vues peuvent être persistantes. Elles sont utilisées pour indexer et requêter les données, produire des listes de données, extraire et filtrer des données, réduire le volume de données, ou pour effectuer des calculs.

Les vues sont propres à un bucket. Une vue ne peut pas cibler plusieurs bucket. Dans Couchbase on distingue les vues de développement et les vues de production. Les premières servent à concevoir une vue avant de l'appliquer au cluster afin de vérifier qu'elle n'engendre pas de problèmes. Les mises à jour des vues se fait de façon incrémentale, il est possible de définir la façon dont la vue se met à jour en fonction de plusieurs paramètres : nombre d'éléments mis à jour, par défaut 5000, et l'intervalle en millisecondes, 5000 également. Seules les vues de productions sont mises à jour automatiquement. Il est cependant possible de forcer la mise à jour des index avec le paramètre `stale`.

- `stale = ok`, donne la réponse la plus rapide sans mise à jour de l'index sur la vue,
- `stale = false`, l'index est mis à jour en premier, les résultats sont donnés ensuite.
- `stale = update_after`, c'est le paramètre par défaut, l'index de la vue se met à jour après une requête sur celle-ci.

Les définitions de vues sont stockées dans des documents de conception (design document), avec les fonctions de mapReduce. Dans le document de conception sont stockées les options de mise à jour des vues. Les documents de conceptions sont créés dans l'interface graphique ou par des requêtes http via curl.

Lorsqu'une vue est créée, l'interface montre les champs disponibles sur un document, crée une fonction de map par défaut, celle qui émet la clé du document et aucune donnée. Interface en annexe.

```
function (doc, meta) {  
    emit(meta.id, null);  
}
```

Il est possible de filtrer et agréger les résultats, par défaut, pas de fonction de reduce, le `stale` est à `false` : mise à jour de l'index forcée.

REDUCE PERSONNALISE ET REREDUCE

Dans Couchbase, il est possible d'écrire ses fonctions de reduce, il faut cependant prêter attention à l'option rereduce de cette fonction. En effet lorsque Couchbase traite un volume important de données, Couchbase fait des calculs préliminaires et stocke ses résultats dans une structure en arbre B. Ainsi Couchbase appliquera le reduce sur un premier jeu de données qu'il stockera dans l'arbre B, puis passera aux jeux de données suivants. Si plusieurs niveaux d'arbre B sont créés alors il faut rappeler le reduce.

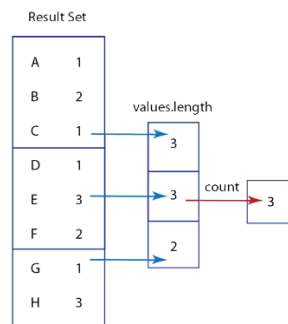


Schéma du rereduce dans Couchbase

PREMIERE IMPLEMENTATION : UN SIMPLE COUNT(*)

Il existe une fonction de count prédéfinie, mais pour appliquer le principe du MapReduce je vais le faire manuellement. Le résultat de la vue doit nous donner le nombre de collisions (documents) dans la base. La fonction de map sera la suivante :

```
function (doc, meta) {  
    emit (1, doc.unique_key);  
}
```

Ici nous émettons la clef du document, propriété qui nous servira à compter son occurrence. Nous sommes sûrs que la clef existe. J'aurais pu tout aussi bien choisir l'id donné par Couchbase, doc.id ou passer tout le document.

La fonction de reduce est la suivante :

```
function (keys, values, rereduce) {  
    if (rereduce) { //le rereduce est propre au système, comme expliqué plus haut.  
        return sum(values);  
    }  
    else {  
        return values.length;  
    }  
}
```

Le résultat obtenu est 763968, soit le nombre de collisions. Résultat en annexe.

DEUXIEME IMPLEMENTATION : OBTENIR LE NOMBRE D'ACCIDENTS MORTELS PAR QUARTIERS

Je vais créer une vue de développement affichant le nombre d'accidents mortels par quartier. La fonction de map va nous servir à trier les données sur le critère accident mortel. J'émetts les données avec comme clé le nom du quartier pour le groupement de Reduce.

Après plusieurs tentatives où le résultat ne s'affichait pas regroupé, contrairement à CouchDB, il s'avère que dans le filtrage des résultats il faut cocher la case « group », en plus de « reduce ». La fonction de reduce reste inchangée, la fonction map devient :

```
function (doc, meta) {
  if(doc.number_of_persons_killed > 0) {
    emit (doc.borough , doc.unique_key);
  }
}
```

Avec pour résultat :

Key	Value
undefined	252
"BRONX"	97
undefined	235
"BROOKLYN"	124
undefined	191
"MANHATTAN"	39
undefined	
"QUEENS"	
undefined	
"STATEN ISLAND"	
undefined	

Résultat du MapReduce

VERIFICATION DES RESULTATS EN UTILISANT LE LANGAGE N1QL

Avec N1QL, effectuer le count se fait comme en SQL.

```
select count(*) from NYC WHERE TONUMBER(number_of_persons_killed) > 0;
```

```
cbq.exe> select count(*) from NYC WHERE TONUMBER(number_of_persons_killed) > 0;
{
  "requestID": "d1c796f1-bcb2-4f54-b880-34418d4f9bb0",
  "signature": {
    "$1": "number"
  },
  "results": [
    {
      "$1": 938
    }
  ],
  "status": "success",
  "metrics": {
    "elapsedTime": "50.7949053s",
    "executionTime": "50.7949053s",
    "resultCount": 1,
    "resultSize": 33
  }
}
```

Sortie de la requête N1QL

L'agrégation en N1QL

```
select borough, count(*) from NYC WHERE TONUMBER(number_of_persons_killed)
> 0 GROUP BY borough;
```

```

cbq.exe> {om NYC WHERE TONUMBER(number_of_persons_killed) > 0 GROUP BY borough;
{
  "requestID": "7ac72537-7a83-4e83-8437-974a50db96c4",
  "signature": {
    "$1": "number",
    "borough": "json"
  },
  "results": [
    {
      "$1": 124,
      "borough": "MANHATTAN"
    },
    {
      "$1": 191,
      "borough": "QUEENS"
    },
    {
      "$1": 39,
      "borough": "STATEN ISLAND"
    },
    {
      "$1": 235,
      "borough": "BROOKLYN"
    },
    {
      "$1": 252,
      "borough": "BRONX"
    }
  ],
  "status": "success",
  "metrics": {
    "elapsedTime": "52.8590234s",
    "executionTime": "52.8590234s",
    "resultCount": 6,
    "resultSize": 372
  }
}
}

```

Sortie de la requête N1QL

ETUDE ET DISCUSSION DES RESULTATS

Accéder aux données pour nos deux requêtes prend environ 55 secondes en exécutant une requête directement par cbq.exe. En fait, Couchbase crée une vue à la volée pour présenter les résultats et la détruit. Effectuer la même requête immédiatement après donnera le résultat avec le même coût de temps. De même, en créant une vue de développement, lorsque nous l'appliquons aux données, elle prend beaucoup de temps à se générer. Cependant, lorsque la vue est passée en production, en appuyant sur le bouton « Publish » de l'interface, le résultat de la requête sera instantané.

Pour accéder aux résultats disponibles par une vue, en dehors de l'interface web de Couchbase, nous devons utiliser l'API Rest via Curl, avec le format suivant : GET /[bucket-name]/_design/[ddoc-name]/_view/[view-name]. Ce qui donne comme url d'accès:

`http://localhost:8092/NYC/_design/AccidentQuartier/_view/AccidentQuartier`

La réponse est ici immédiate.

```

C:\Users\Sylvain>curl -v http://localhost:8092/NYC/_design/AccidentQuartier/_vie
w/AccidentQuartier
* Trying ::1...
* Trying 127.0.0.1...
* Connected to localhost (127.0.0.1) port 8092 (#0)
> GET /NYC/_design/AccidentQuartier/_view/AccidentQuartier HTTP/1.1
> Host: localhost:8092
> User-Agent: curl/7.46.0
> Accept: */*
<
< HTTP/1.1 200 OK
< Transfer-Encoding: chunked
< Server: MochiWeb/1.0 (Any of you quaid's got a smint?)
< Date: Sat, 19 Mar 2016 18:10:19 GMT
< Content-Type: text/plain; charset=utf-8
< Cache-Control: must-revalidate
<
{
  "rows": [
    {
      "key": null, "value": 938
    }
  ]
}
* Connection #0 to host localhost left intact
C:\Users\Sylvain>

```

Réponse de la vue pour l'équivalent du Count(*)

Un utilisant N1QL, il faut créer un index, qui se basera soit sur un index global secondaire (GSI), soit sur une vue. Sera alors utilisé les mots-clefs USING GSI ou USING VIEW à la fin de la commande CREATE INDEX. Les vues avec MapReduce ne peuvent pas être utilisées. Lorsque l'index est créé, derrière, une vue pour N1QL est établie. L'index sera créé de la façon suivante, sur les arrondissements :

```
CREATE INDEX quartier ON NYC(borough) USING GSI;
```

La requête ramenant les résultats groupés par arrondissements sera légèrement plus rapide, 48 secondes au lieu de 55. Ce qui n'équivaut pas les résultats des vues MapReduce d'après mes expérimentations.

Comme nous l'avons abordé, les vues mapReduce sont pré-calculées, ce qui n'est pas le cas avec les requêtes N1QL. Les requêtes REST ont donc une réponse immédiate contrairement à celles-ci. Dans certains cas, les requêtes N1QL pourront être performantes, dans d'autres cas beaucoup moins.

Dans ce cas pourquoi utiliser N1QL ? Le langage N1QL permet d'exécuter des requêtes ad hoc sans devoir implémenter une vue. De plus le langage utilise des mots proches de SQL, de fait, pour une personne n'ayant pas connaissance des spécificités des systèmes de base de données documentaires et du paradigme clé valeur, celle-ci peut facilement interroger les données. Ces requêtes sont par ailleurs portables, puisqu'avec un connecteur ODBC, elles pourront être exécutées par un logiciel tiers.

Les vues quant à elles permettent de délivrer les résultats avec des performances accrues. C'est grâce au pré calcul implémenté en JavaScript par les fonctions de Map et de Reduce que cela est possible. Elles ont ainsi l'avantage de proposer une structure formatée pour un service consommateur.

Nous venons de voir le panel des possibilités pour interroger les données dans Couchbase. Les différentes requêtes permettent ainsi de récupérer des jeux de données à analyser par un programme client. Mais pour aller plus loin, comment pouvons-nous mettre en place une structure permettant la recherche d'informations avec classement dans le système documentaire ?

V. Indexation de Couchbase par Elasticsearch

Pour intégrer un service d'indexation et de recherche de données plain text au cluster Couchbase, il est possible d'interfacer un serveur Elasticsearch. Dans le cadre du projet, j'ai installé les serveurs sur la même machine, mais évidemment l'idéal est de dédier les environnements sur des machines différentes. Elasticsearch s'intègre complètement dans un environnement Couchbase. En effet, le serveur Elasticsearch sera rajouté en tant que cluster distant recevant une réplication du cluster Couchbase. Ainsi seront envoyés vers le serveur Elasticsearch les documents contenus dans les buckets choisis pour être répliqués. Elasticsearch recevra toutes les mises à jour des documents contenus dans les buckets répliqués, il pourra ainsi les indexer et offrir un service de recherche.

1. Installation et configuration du serveur Elasticsearch

Avant de procéder à l'installation d'Elasticsearch, il faut vérifier la compatibilité de plugin avec la version du serveur Couchbase et avec la version du serveur Elasticsearch. Dans le cadre de mon installation, Couchbase v4.x, il faut récupérer la version 2.2.1.2 du plugin, compatible avec la version 2.1.1 d'Elasticsearch. Pour installer le serveur, il faut télécharger le zip contenant les fichiers de l'application, puis installer le serveur web, comme vu dans le cadre du cours. Pour installer le plugin, qui est l'équivalent de la rivière MongoDB mais pour Couchbase, il faut lancer la commande suivante :

```
plugin install https://github.com/couchbaselabs/elasticsearch-transport-couchbase/releases/download/v2.2.1.2/elasticsearch-transport-couchbase-2.2.1.2.zip
```

FICHIER DE CONFIGURATION

Une fois le plugin installé, il faut éditer le fichier de configuration elasticsearch.yml pour rajouter certains éléments nécessaires pour interfacier le serveur Elasticsearch avec Couchbase :

```
couchbase.username: admin
couchbase.password: password
couchbase.maxConcurrentRequests: 1024
cluster.name: ElasticSearch
node.name: Couchbase
```

Les identifiants seront demandés par Couchbase pour se connecter au serveur Elasticsearch lors de la création du cluster distant. Le paramètre maxConcurrentRequests définit la limite haute des requêtes simultanées pouvant être traitées. Les deux paramètres suivants permettent de définir un nom au cluster et au nœud du serveur Elasticsearch. Dans le cas contraire des noms générés aléatoirement seraient appliqués.

Une fois la configuration établie, j'ai démarré le serveur Elasticsearch. Des erreurs sont apparues, que je suis allé consulter dans le fichier de log situé dans le répertoire logs du répertoire d'installation du serveur. Après recherches, il s'avère que pour cette version du plugin, il faut éditer le fichier de gestion de la sécurité dans Java et d'ajouter des exceptions pour les binaires du plugin.

Le fichier à éditer se situe dans le répertoire Java indiqué comme JAVA_HOME dans les variables systèmes Windows, dans mon cas il est situé au chemin suivant :

C:\Program Files\Java\jdk1.8.0_77\jre\lib\security.

A la fin du fichier java.policy, il faut rajouter les lignes suivantes.

```
49
50 grant codeBase "file:/E:/NFE204DATA/cbelastic/plugins/transport-couchbase/" {
51     permission javax.security.auth.AuthPermission "modifyPrincipals";
52     permission javax.security.auth.AuthPermission "modifyPrivateCredentials";
53     permission javax.security.auth.AuthPermission "setReadOnly";
54     permission java.lang.RuntimePermission "setContextClassLoader";
55     permission java.net.SocketPermission ".*", "listen,resolve";
56     permission java.lang.reflect.ReflectPermission "suppressAccessChecks";
57 };
58
59
```

Modification du fichier de gestion de la sécurité pour les programmes utilisant Java

Une fois le fichier modifié, le serveur Elasticsearch démarrera sans erreur.

CREATION DE L'INTERFACE ENTRE COUCHBASE ET ELASTICSEARCH

Lors de l'installation du plugin pour Couchbase, un modèle par défaut permettant d'indexer les documents envoyés par Couchbase est disponible. On le déploie avec curl par la commande suivante :

```
curl -XPUT http://192.168.0.10:9200/_template/couchbase -d
@E:\NFE204DATA\cbelastic\plugins\transport-
couchbase\couchbase_template.json
```

Le système doit répondre : {"acknowledged":true}

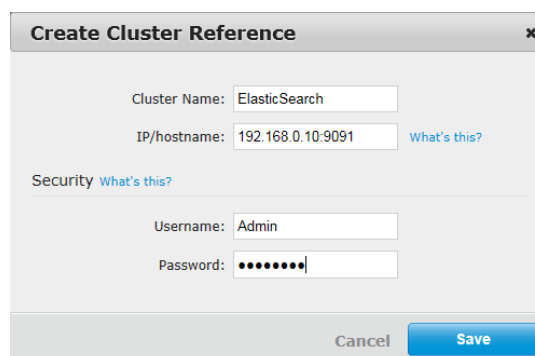
On prépare ensuite un index sur le serveur d'indexation pour stocker les données d'un bucket qui lui seront envoyés, en écrivant le nom de bucket en minuscule. Pour le bucket NYC, la commande sera :

```
curl -XPUT http://192.168.0.10:9200/nyc -d '{
  "settings" : {
    "number_of_shards" : 1,
    "number_of_replicas" : 0
  }
}'
```

La commande prend comme options des paramètres au format JSON ou YAML, où l'on spécifie le nombre de shards et de répliques. Si cette option n'est pas incluse, alors ce seront les éléments spécifiés dans le fichier de config d'Elasticsearch qui seront pris. Il est également possible de passer par l'interface web, où l'on pourra directement saisir le nombre de shards et de répliques disponibles.

MISE EN ROUTE DE LA REPLICATION DU CLUSTER COUCHBASE

A partir de l'interface web de Couchbase, sous l'onglet XCDR permettant de configurer la réplication de cluster, on ajoute une nouvelle référence en cliquant sur le bouton « create cluster reference ». Il faut saisir l'adresse IP du serveur distant, le nom du cluster, et les paramètres d'authentification. En saisissant l'adresse de la boucle locale 127.0.0.1, je ne parvenais pas à établir la connexion sur le port utilisé par ElasticSearch, le port 9091. Après recherches, la connexion ne pouvait pas s'établir par la boucle locale avec cette version de Couchbase, il fallait saisir l'adresse de mon ordinateur sur le réseau local.



Création de la référence vers le nouveau cluster

Une fois la connexion établie, on lance la réplication en cliquant sur le bouton « Create Replication ». Il faut saisir le bucket source et le bucket cible, qui est l'index précédemment créé sous Elasticsearch. Ensuite dans les paramètres avancés, il faut choisir la version 1 du protocole de réplication, qui établit une réplication avec le protocole REST. C'est ce qui est conseillé dans la documentation de Couchbase pour interfacer Elasticsearch. La version 2 du protocole est utilisée pour la réplication entre clusters Couchbase, celle-ci est basée sur un protocole REST mem-cached appelé XMEM mode pour des performances de réplication plus élevées.

Un fois validé, la réplication débute. Nous voyons que le déclencheur est spécifié sur l'évènement Changed et que la réplication est en cours.

Replications

REMOTE CLUSTERS						Create Cluster Reference
Name	IP/hostname					
ElasticSearch	192.168.0.10:9091					Edit Delete

ONGOING REPLICATIONS						Create Replication
Bucket	Protocol	From	To	Status	When	
NYC	Version 1	this cluster	bucket "nyc" on cluster "ElasticSearch"	Replicating	on change	Edit Delete

Réplication Cross Data Center en cours

Puis sur l'interface d'Elasticsearch, nous voyons les documents répliqués arriver par lots. Pour finir nous avons 764 993 documents qui ont été reçus.

The screenshot shows the Elasticsearch web interface. At the top, the URL is http://192.168.0.10:9200/. The main navigation bar includes 'Aperçu', 'Index', 'Navigateur', 'Recherche Structurée [+]', and 'Autres requêtes [+]'.

The 'Aperçu du cluster' section shows the 'nyc' index with a size of 356Mi (356Mi) and 764 993 documents (766 885). Below this, the Couchbase cluster status is shown as 'Santé du cluster: green (1 1)'.

Index nyc du nœud Couchbase, possédant 764 993 documents, 1 shard, 0 réplique.

Afin de vérifier que l'indexeur interprète correctement les documents envoyés par Couchbase, nous pouvons aller dans les informations d'état du cluster, nous y voyons que les propriétés liées aux documents JSON du bucket NYC ont bien été récupérées. A gauche de l'interface nous voyons les champs disponibles préfixés de « doc ». Le type couchbaseDocument est créé par le plugin Elasticsearch sous la forme d'un objet possédant des propriétés, les champs du document. Le mapping est par défaut dynamique pour tous les champs de l'objet de type couchbaseDocument. Il est possible de créer des types pour différents documents contenus dans Couchbase, et d'associer un mapping spécifique pour chacun de ces types.

Tous les index	Recherche sur 5 des 5 shards. 764993 résultats. 0.017 secondes				
INDEX	_index	_type	_id	_score	meta.rev
beer-sample	nyc	couchbaseDocument	189260	1	1-143b1957d20100000000000000000000 0
nyc	nyc	couchbaseDocument	101457	1	1-143b1946603b00000000000000000000 0
TYPES	nyc	couchbaseDocument	275102	1	1-143b196cf9c100010000000000000000 0
default	nyc	couchbaseDocument	181195	1	1-143b195669bc00030000000000000000 0
couchbaseCheckpoint	nyc	couchbaseDocument	297385	1	1-143b19772a3400000000000000000000 0
couchbaseDocument	nyc	couchbaseDocument	3203728	1	1-143b198bc32200000000000000000000 0
CHAMPS	nyc	couchbaseDocument	104546	1	1-143b194a385c00020000000000000000 0
▶ doc.borough	nyc	couchbaseDocument	244944	1	1-143b19627cd400000000000000000000 0
▶ doc.contributing_factor_vehicle_1	nyc	couchbaseDocument	1005972	1	1-143b1945068100000000000000000000 0
▶ doc.contributing_factor_vehicle_2	nyc	couchbaseDocument	3150774	1	1-143b1982664b00000000000000000000 0
▶ doc.contributing_factor_vehicle_3	nyc	couchbaseDocument	3160144	1	1-143b1983f89600000000000000000000 0
▶ doc.contributing_factor_vehicle_4	nyc	couchbaseDocument	217135	1	1-143b195ce89f00010000000000000000 0
▶ doc.contributing_factor_vehicle_5	nyc	couchbaseDocument	21011	1	1-143b195bca7100000000000000000000 0
▶ doc.cross_street_name	nyc	couchbaseDocument	3165727	1	1-143b1984fc0f00020000000000000000 0
▶ doc.date	nyc	couchbaseDocument	1034134	1	1-143b19494d9f00000000000000000000 0
▶ doc.latitude	nyc	couchbaseDocument	3149488	1	1-143b198219fe00000000000000000000 0
▶ doc.location.latitude	nyc	couchbaseDocument	185980	1	1-143b195759b700010000000000000000 0
▶ doc.location.longitude	nyc	couchbaseDocument	30188	1	1-143b197941e700000000000000000000 0
▶ doc.location.needs_recoding	nyc	couchbaseDocument	184666	1	1-143b195709aa00020000000000000000 0
▶ doc.longitude	nyc	couchbaseDocument	3243307	1	1-143b19936fd000010000000000000000 0
▶ doc.number_of_cyclist_injured	nyc	couchbaseDocument	128779	1	1-143b194e009200000000000000000000 0
▶ doc.number_of_cyclist_killed	nyc	couchbaseDocument	323010	1	1-143b1990e6be00000000000000000000 0
▶ doc.number_of_motorist_injured	nyc	couchbaseDocument	3158263	1	1-143b1983a68200000000000000000000 0
	nyc	couchbaseDocument	315510	1	1-143b1983279a00020000000000000000 0

Information de l'état du cluster

REQUETES DE RECHERCHE PAR L'INTERFACE

Notre réplication établie, nous pouvons commencer à exécuter des recherches par le serveur Elasticsearch. La requête `{"query":{"query_string":{"query":"doc.borough:bronx"}}}` retournera tous les documents trouvés selon ces critères. Cependant, seules les métadonnées des documents seront affichées, pas le contenu. Ce qui est un comportement standard. Le moteur renvoie l'id du document qui fait partie des métadonnées. A charge de l'application client d'exécuter une requête sur la base pour récupérer les champs désirés. Dans notre cas, pour tout de même afficher le contenu des documents dans le résultat de la recherche, il faut modifier la configuration de la réplication dans le fichier de template du plugin Couchbase.

Dans le mapping entre Couchbase et Elasticsearch, il faut rajouter les champs sources disponibles. Par défaut seules les métadonnées étaient incluses, il faut rajouter `doc.*` dans la propriété multi-valuée `includes`.

```
"_default_" : {
  "_source" : {
    "includes" : ["meta.*","doc.*"]
  },
  "properties" : {
    "meta" : {
      "type" : "object",
      "include_in_all" : false
    }
  }
}
```

Une fois le document édité, il est nécessaire de renvoyer le template à Elasticsearch par la commande vue plus haut, puis de supprimer et de recréer l'index. Cette fois, la requête de recherche ramènera bien le contenu des documents dans le résultat.

search

POST

```

{
  "query": {
    "query_string": {
      "query": "doc.borough:bronx"
    }
  }
}

```

Requête

Valider le JSON

Pretty

Transformation des résultats

Répétition automatique de la requête

Options d'affichage

☒ En JSON brut
 ☐ En graphe
 ☐ En tableau

```

{
  "total": 1,
  "successful": 1,
  "failed": 0
},
{
  "hits": {
    "total": 73946,
    "max_score": 3.3353167,
    "hits": [
      {
        "_index": "nyc",
        "_type": "couchbaseDocument",
        "_id": "314083",
        "_score": 3.3353167,
        "_source": {
          "meta": {
            "rev": "1-143b1980aa9200000000000000000000",
            "flags": 0,
            "expiration": 0,
            "id": "314083"
          },
          "doc": {
            "date": "2014-04-02T00:00:00",
            "number_of_motorist_injured": 0,
            "number_of_motorist_killed": 0,
            "unique_key": "314083",
            "latitude": "40.8609971",
            "number_of_cyclist_killed": 0,
            "on_street_name": "EAST 188 STREET",
            "borough": "BRONX",
            "number_of_persons_killed": 0,
            "number_of_cyclist_injured": 0,
            "zip_code": "10458",
            "vehicle_type_code1": "PASSENGER VEHICLE",
            "number_of_pedestrians_injured": 1,
            "contributing_factor_vehicle_1": "Unspecified",
            "number_of_pedestrians_killed": 0,
            "number_of_persons_injured": 1,
            "location": {
              "needs_recoding": false,
              "latitude": "40.8609971",
              "longitude": "-73.896425"
            }
          }
        }
      }
    ]
  }
}

```

Résultat de la recherche dans l'index nyc

Pour tester notre moteur de recherche nous allons effectuer une requête un peu plus complexe. La requête suivante permettra de rechercher les accidents mortels ayant eu lieu dans le Bronx, avec une voiture de sport, avec un conducteur en état d'ébriété.

```

"query": "doc.borough:bronx AND doc.vehicle_type_code1:SPORT* AND
doc.number_of_persons_killed : [1 TO 10] AND
doc.contributing_factor_vehicle_1: Alcohol Involvement"

```

Parmi les 764 993 documents nous n'obtenons qu'un seul résultat, une collision ayant eu lieu sur la rue GRAND CONCOURSE à 1h21 du matin.

ETUDE ET DISCUSSION DES RESULTATS

Nous venons d'établir la réplcation des données contenues dans Couchbase dans un serveur d'indexation permettant d'effectuer des recherches. Cette installation complète efficacement les fonctionnalités du cluster Couchbase en effet, nous avons vu qu'il était possible d'interroger Couchbase par son langage de requête N1QL ou par l'intermédiaire de vues Couchbase créées sur mesure. Ces deux possibilités sont ouvertes sur le protocole http, les vues ayant un temps de réponse très rapide mais qui nécessitent leur mise en place au préalable, ou contraire des requêtes N1QL qui permettent une interrogation à la volée au détriment de la vitesse.

La mise en place du serveur Elasticsearch va concilier les deux, rapidité et flexibilité.

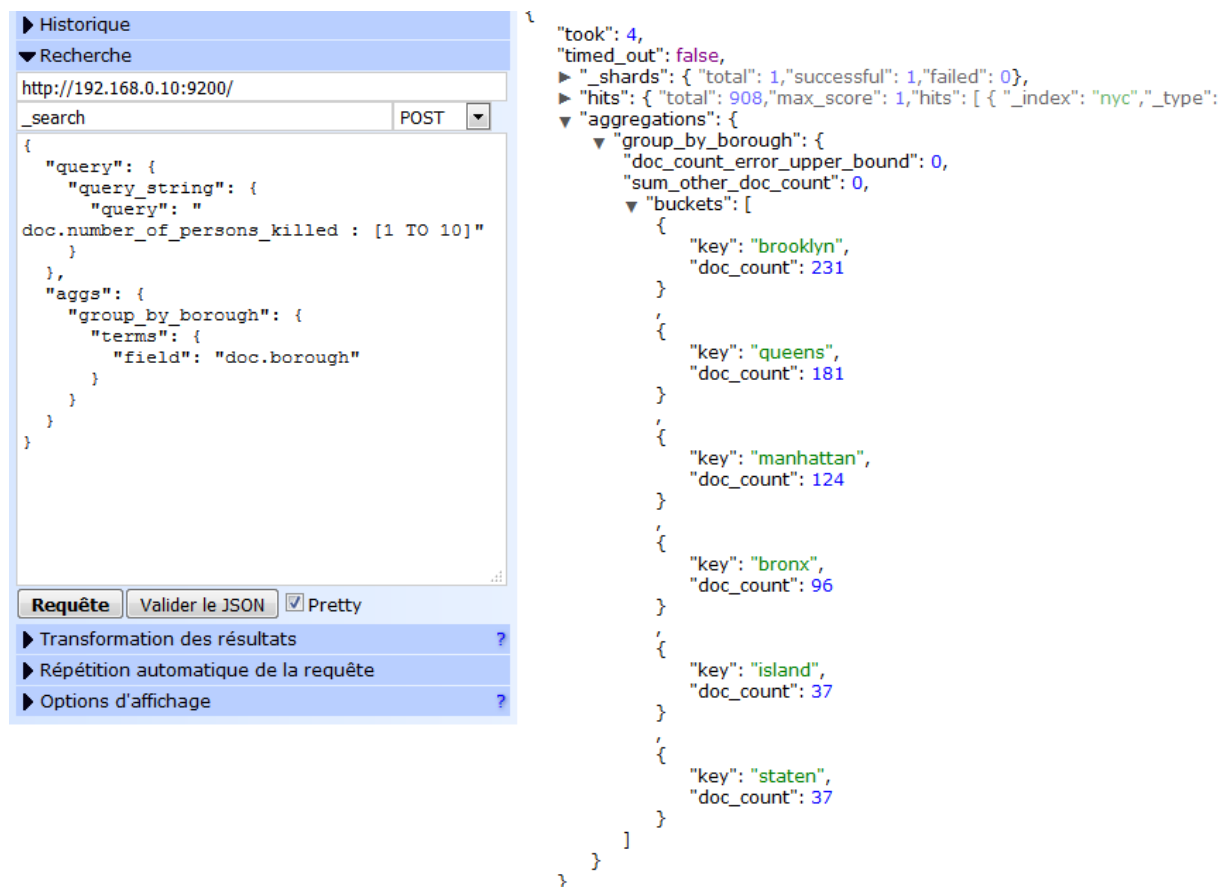
Reprenons l'exemple de la requête suivante : le nombre d'accidents mortels par arrondissements.

Si une vue est créée avec l'agrégation dans couchbase, la réponse est immédiate. Alors qu'avec une requête N1QL, le temps de réponse sera de plus de 50 secondes. Essayons de faire la même requête mais au serveur Elasticsearch.

La syntaxe permettant d'ajouter une agrégation sera la suivante :

```
"aggs": {
  "group_by_borough": {
    "terms": {
      "field": "doc.borough"
    }
  }
}
```

Dans la requête http au paramètre query, nous ajoutons après query_string, l'argument aggs qui contient le nom de l'agrégation les champs sur lesquels elle est appliquée. En SQL nous aurions écrit GROUP BY Borough.



The screenshot shows a web interface for a search engine. On the left, there's a 'Recherche' (Search) section with a URL bar showing 'http://192.168.0.10:9200/' and a search box containing '_search'. Below the search box is a 'POST' button. The main area displays a JSON query body:

```
{
  "query": {
    "query_string": {
      "query": "
doc.number_of_persons_killed : [1 TO 10]"
    }
  },
  "aggs": {
    "group_by_borough": {
      "terms": {
        "field": "doc.borough"
      }
    }
  }
}
```

Below the query body are buttons for 'Requête', 'Valider le JSON', and 'Pretty'. On the right, the results are displayed in a tree view. The root object is:

```
{
  "took": 4,
  "timed_out": false,
  "_shards": { "total": 1, "successful": 1, "failed": 0 },
  "hits": { "total": 908, "max_score": 1, "hits": [ { "_index": "nyc", "_type": "
  "aggregations": {
    "group_by_borough": {
      "doc_count_error_upper_bound": 0,
      "sum_other_doc_count": 0,
      "buckets": [
        {
          "key": "brooklyn",
          "doc_count": 231
        },
        {
          "key": "queens",
          "doc_count": 181
        },
        {
          "key": "manhattan",
          "doc_count": 124
        },
        {
          "key": "bronx",
          "doc_count": 96
        },
        {
          "key": "island",
          "doc_count": 37
        },
        {
          "key": "staten",
          "doc_count": 37
        }
      ]
    }
  }
}
}
```

Requête de recherche avec agrégation

Dans la fenêtre de résultat nous voyons que la requête a été exécutée en 4 millisecondes. Comparé aux 55 secondes demandées par la requête directe N1QL, c'est plus de 10.000 fois plus rapide.

Seul bémol, les résultats diffèrent légèrement du résultat exact, pour Brooklyn le résultat attendu aurait dû être 235 au lieu de 213. La propriété portant la valeur de l'arrondissement Staten Island a été coupé en deux termes distincts. Pour éviter que les termes soient coupés, il faut modifier le mapping du champ dans le fichier de configuration du plugin qui ici a été réglé sur dynamique. Le

champ Borough a été traité comme texte et découpé en termes au lieu d'être traité comme une chaîne de caractères. Quant à la légère différence de résultat, il faudrait je pense pousser plus loin le paramétrage du moteur pour identifier la cause. Cette nécessité de paramétrage fin du moteur pour l'aligner avec les sources de données analysées est propre à toutes les implémentations de moteurs de recherche.

2. Configuration du mapping Elasticsearch

Nous venons de voir comment répliquer les données d'un bucket dans le serveur d'indexation. Nous avons vu que la configuration de la rivière de base est assez simple. C'est parce qu'un ensemble de paramètres sont inclus par défaut. Cependant il est possible de créer une configuration plus avancée afin de paramétrer le fonctionnement du serveur d'indexation selon nos souhaits. Pour illustrer les possibilités offertes par la configuration du serveur, je vais changer le jeu de données pour un nouveau ensemble de documents qui se prêtera plus à des manipulations sur du texte.

JOB SEARCH

Toujours sur la base de données Open Data de la ville de New-York, j'ai récupéré les annonces de recherche d'emploi. Je n'ai pas établi de processus spécifique pour récupérer les données, le principe a déjà été démontré au point III.

La liste des offres d'emploi est disponible au format JSON à cette adresse :

`https://nycopendata.socrata.com/City-Government/NYC-Jobs/kpav-sd4t.json`

J'ai donc récupéré les annonces dans un document json appelé NYCJobs.json. Pour envoyer les données dans Couchbase, il faut découper nos documents en autant de fichiers distincts comme vu précédemment. J'ai utilisé le même script python. Il a cependant fallu l'adapter, les chaînes de caractères contenues dans les annonces posaient un problème d'encodage. Il a fallu passer de l'UTF-8 à un encodage latin. Voici la ligne de script modifiée, en ayant importé la bibliothèque codecs :

```
data = json.load(codecs.open(str(path+filename), encoding="ISO-8859-1"))
```

Une fois le script lancé, j'ai obtenu 2101 annonces distinctes, l'export en contenait deux fois plus mais elles étaient quasiment toutes doublées, le découpage par la propriété job_id a retiré les doublons. J'ai procédé à la création d'un nouveau bucket dans Couchbase appelé JOBS et d'un index dans elasticsearch, jobs également. L'import dans Couchbase s'est fait par l'utilitaire cbdocloader déjà évoqué.

Les documents JSON des annonces d'emplois contiennent un certain nombre de propriétés, je ne vais pas les décrire, ce n'est pas l'objectif ici. J'évoque seulement celles qui nous intéressent :

- `business_title` : titre de l'annonce
- `job_description` : description de l'emploi

La procédure de création de la réplication vers Elasticsearch est la même. Je ne détaille pas. Je vais au contraire m'attacher à décrire les possibilités offertes par le mapping et ensuite par la configuration de l'analyseur.

CONFIGURATION DU MAPPING

Nous avons vu précédemment que dans le fichier `couchbase_template.json` sont spécifiés les propriétés du mapping entre les documents envoyés par Couchbase et les documents insérés dans Elasticsearch. Je vais détailler la configuration.

La propriété `"template"` : `"*"` indique que ce qui est défini après, s'applique pour tous les index. Si l'on remplace `*` par le nom d'un index alors le template ne s'appliquera que pour cet index en particulier. Cela permet de créer des comportements différents pour chaque réplication vers l'indexeur.

La propriété `order` permet de gérer l'ordre dans lequel sont appliqué les templates lorsque ceux-ci s'appliquent sur un même index.

Le mapping se fait par types de documents, la propriété `_default_` indique que le mapping sera fait pour tous les documents. Il est d'ores et déjà possible de restreindre le mapping sur les documents de Couchbase en remplaçant cette valeur par `couchbaseDocument`.

La propriété `_source` indique ce qui sera conservé dans l'indexeur comme champ source disponible pour un document, c'est-à-dire non traité par l'analyseur, conservé intact afin de pouvoir le consulter. Comme nous l'avons vu par défaut `source` contient `"includes" : ["meta.*"]`. Ce qui signifie que seules les métadonnées des documents seront affichées dans les résultats d'une requête de recherche. Pour rajouter tous les champs dans les résultats de recherche il faut modifier cette ligne en `"includes" : ["meta.*", "doc.*"]`.

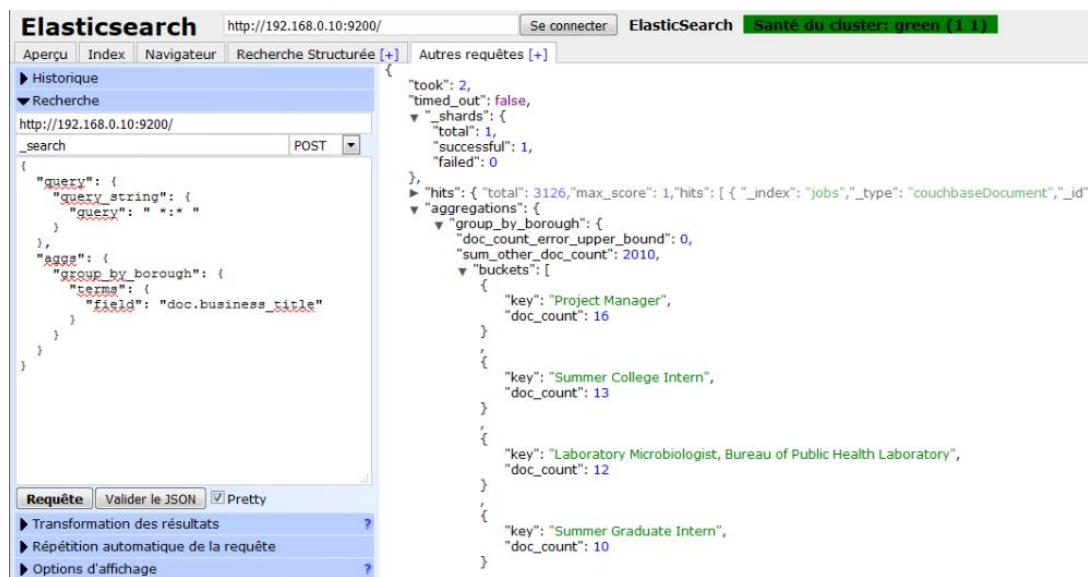
J'ai rajouté la propriété `doc` pour les documents Couchbase afin de pouvoir réaliser le mapping. Voir annexe. La propriété `dynamic` est passé à `false` pour indiquer que le mapping est manuel. Ensuite il faut spécifier les champs qui seront analysés. J'ai repris les deux champs mentionnés auparavant, `business_title` et `job_description`. Les deux sont marqués en tant que chaîne de caractères. Pour `business_title`, la propriété `index` est sur `not_analyzed`. Cela signifie qu'aucun analyseur ne fera de traitement sur la propriété `business_title`. Ainsi les valeurs ne seront pas tokenisées. C'est utile lorsque l'on souhaite qu'une chaîne de caractère ne soit pas découpée pour garder son sens.

Je souhaite regrouper toutes les offres d'emplois sur le nom du job. Si le champ est analysé, alors il sera tokenisé, dans ce cas le regroupement se fera sur chaque partie de mots, comme montré dans la capture ci-dessous : bureau, health, director etc... Ce qui n'a pas de sens. Vous remarquerez que des mots inutiles apparaissent comme des tokens : of, and, etc. Nous verrons comment gérer les stopwords ensuite.



Regroupement sur des tokens

Lorsque l'on ajoute l'entrée index : not_analyzed pour le champ business_title, voici le résultat avec la même requête, après recréation de l'index.



Regroupement sur la chaîne complète

Par défaut le système retourne 10 valeurs pour un regroupement. Pour retourner plus de valeurs, il est possible de rajouter la propriété size en dessous de field, dans les termes de l'agrégation. A zéro, toutes les valeurs sont affichées.

Le mapping permet d'autres types de configuration, il est par exemple possible de spécifier ou de forcer le type d'un champ en un entier, un géocode, une date. Il est également possible aussi pour chaque propriété de dire si elle sera stockée ou non par la propriété store. Valable pour le cas où source est à false et pour des cas bien précis. Pour traiter les valeurs contenues dans les propriétés des documents, il faut créer un analyseur. Ce que nous allons voir.

CREATION D'UN ANALYSEUR

Un analyseur peut être déclaré dans un fichier de configuration, puis associé à un index lors de la création de celui-ci par une requête Curl. Je déclare l'analyseur personnalisé en créant un fichier appelé JobAnalyzer.json dans le répertoire config. L'analyseur est spécifié au moment de la création de l'index par la commande :

```
curl -XPUT http://192.168.0.10:9200/jobs -d
@E:\NFE204DATA\cbelasticsearch\config\JobAnalyzer.json
```

La commande suivante permet de tester un analyseur d'un index en lui envoyant une chaîne :

```
curl -XGET http://192.168.0.10:9200/jobs/_analyze?analyzer=JobAnalyzer -d
"this is a test"
```

TOKENIZATION

Il existe des analyseurs par défaut, dont l'analyseur standard, english, french, etc. Il est possible de tester au préalable un analyseur avant d'injecter les données dans un index. Voici comment on procède par une requête Curl, sur l'index jobs :

```
curl -XGET http://192.168.0.10:9200/jobs/_analyze?analyzer=standard -d
"this is a test"
```

La console donnera 4 tokens alphanumériques :

```
{"tokens":[{"token":"this","start_offset":0,"end_offset":4,"type":"<ALPHANUM>","position":0}, {"token":"is","start_offset":5,"end_offset":7,"type":"<ALPHANUM>","position":1}, {"token":"a","start_offset":8,"end_offset":9,"type":"<ALPHANUM>","position":2}, {"token":"test","start_offset":10,"end_offset":14,"type":"<ALPHANUM>","position":3}]}
```

Nous voyons que des mots inutiles sont tokenisés. Changeons d'analyseur pour celui dédié au vocabulaire anglais.

```
curl -XGET http://192.168.0.10:9200/jobs/_analyze?analyzer=english -d "this is a test"
```

La console ne donnera plus qu'un seul token alphanumérique :

```
{"tokens":[{"token":"test","start_offset":10,"end_offset":14,"type":"<ALPHANUM>","position":3}]}
```

Les mots vides ont été retirés par l'analyseur english d'Elasticsearch, pour ne garder que le terme test. Les stopwords de l'analyseur English sont les suivants :

a, an, and, are, as, at, be, but, by, for, if, in, into, is, it, no, not, of, on, or, such, that, the, their, then, there, these, they, this, to, was, will, with

Il est possible de créer sa propre liste de stopwords. Le fichier texte est à déposer dans le répertoire config d'Elasticsearch avec un mot sur chaque ligne. Il faut rajouter la ligne "stopwords_path": "stopwords/CustomStopWords.txt" dans le fichier de configuration de l'analyseur.

Ayant placé le terme « test » dans une nouvelle liste de stopwords pour la langue anglaise récupérée sur le site [ranks.nl](http://www.ranks.nl), aucun token n'est créé si j'envoie la même requête.

```
{"tokens":[]}
```


Envoyer pour analyse « this is a test for my indexer » retourne le résultat suivant :

```
{ "tokens": [ { "token": "index", "start_offset": 22, "end_offset": 29, "type": "<ALPHA NUM>", "position": 6 } ] }
```

Seul « indexer » a été tokenisé, en un terme « index ». C'est le stemming qui a normalisé le mot.

STEMMING

Pour mettre en place un stemming personnalisé, j'ai créé un nouvel analyseur appelé JobAnalyzerStem. Il faut retirer la propriété "type": "english" car nous définissons notre propre stemming. L'analyseur english d'Elasticsearch est configuré de façon identique à celui de Lucene (voir annexe pour les filtres contenus), avec une configuration préétablie et prête à l'emploi. Ici nous en créons un nouveau pour l'exemple. Le nouvel analyseur définit une surcharge du stemmer pour forcer la substitution de mots par d'autres au travers de la propriété rules. Il est également possible de retirer certains termes du stemmer en déclarant un nouveau filtre de type keyword_marker et en plaçant les mots dans un tableau.

```
{
  "settings": {
    "analysis": {
      "filter": {
        "custom_stem": {
          // filtre de type surcharge de stemmer pour substituer des mots
          "type": "stemmer_override",
          "rules": [
            "bad=>Lee Van Cleef",
            "good=>Client Eastwood",
            "ugly=>Eli Wallach"
          ]
        },
        // filtre pour ne pas lemmatiser certains mots
        "no_stem": {
          "type": "keyword_marker",
          "keywords": [ "indexer" ]
        },
        // filtre des stopwords
        "customstopwords": {
          "type": "stop",
          "stopwords_path": "stopwords/CustomStopWords.txt"
        }
      },
      "analyzer": {
        "JobAnalyzer": {
          "type": "english",
          // complement a l'analyseur english - une nouvelle liste de stopwords
          "stopwords_path": "stopwords/CustomStopWords.txt"
        },
        "JobAnalyzerStem": {
          "tokenizer": "standard",
          "filter": [
            // Ajout des filtres créés pour cet analyseur
            "customstopwords",
            "lowercase", // filtre natif, mots mis en minuscule
            "custom_stem",
            "no_stem",
            "porter_stem" // filtre natif, algorithme de Porter
          ]
        }
      }
    }
  }
}
```

Fichier de configuration des analyseurs JobAnalyzer.json

Les mots placés dans la propriété `rules` font référence au film de Sergio Leone : *The Good, the Bad and the Ugly*, avec pour chaque personnage, le nom de l'acteur. Le terme « indexer » qui a été lemmatisé en « index » est placé dans la liste des mots ne devant plus être transformés. L'analyse de la phrase « this is a test for the good the bad and the ugly in the indexer » produira donc les tokens suivants:

```
{"tokens":[{"token":"Client Eastwood","start_offset":23,"end_offset":27,"type":"<ALPHANUM>","position":6}, {"token":"Lee Van Cleef","start_offset":32,"end_offset":35,"type":"<ALPHANUM>","position":8}, {"token":"Eli Wallach","start_offset":44,"end_offset":48,"type":"<ALPHANUM>","position":11}, {"token":"indexer","start_offset":56,"end_offset":63,"type":"<ALPHANUM>","position":14}]}
```

Les termes `bad`, `good` et `ugly` ont été remplacés par le nom des acteurs par le filtre `custom_stem`, le terme `indexer` n'a pas été normalisé par le biais du filtre `no_stem` et les stopwords n'ont pas été tokenisés. Ce qui montre le bon fonctionnement de l'analyseur.

MISE EN PLACE DE L'ANALYSEUR POUR LES DONNEES ISSUES DE COUCHBASE

Je vais mapper l'analyseur `JobAnalyzer` de type `english` sur le champ `job_description`. L'analyseur avec le stemming a été configuré pour démontrer le principe. Le premier suffit dans notre cas. La configuration d'un analyseur est un fin paramétrage qui peut nécessiter beaucoup de temps, d'essais et de compromis pour obtenir un résultat satisfaisant.

Avec la commande de création vu précédemment, je recrée l'index `jobs` dans `elasticsearch` puis je renvoie le fichier de config pour la rivière `couchbase` avec la modification ci-dessous :

```
"job_description": {
  "type": "string",
  "analyzer": "JobAnalyzer"
}
```

Il devient possible d'exécuter des requêtes de recherche sur le champ `job_description` au travers de l'interface web d'Elasticsearch. La requête `"query": "doc.job_description : this,is,a,test"` retournera `null`. Les mots sont tous des stopwords ajoutés à l'analyseur pour ce champ. Si la requête est appliquée sur d'autres champs alors des résultats seront retournés.

La requête `"query": "doc.job_description : this,is,a,test,Database,Administration"` donnera des résultats sur les termes `Database` et `Administration` seulement. Il est possible d'avoir le détail du calcul du classement en ajoutant `"explain": true` à la requête. Dans la capture ci-dessous, dans le nœud `explanation` nous voyons le calcul du score qui est composé du score de deux termes, `database` avec un score à 0.695 et `administration` avec un score à 0.161 pour le champ `job_description`.

▼ Recherche

http://192.168.0.10:9200/

_search POST

```
{
  "explain": true,
  "query": {
    "query_string": {
      "query": "doc.job_description :
this, is, a, test, Database, Administration"
    }
  }
}
```

Requête Valider le JSON ☒ Pretty

► Transformation des résultats ?

► Répétition automatique de la requête

► Options d'affichage ?

```
{
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "failed": 0
  },
  "hits": {
    "total": 893,
    "max_score": 0.8564255,
    "hits": [
      {
        "_shard": 0,
        "_node": "F-OAP1wHSWSwniLvEBwoPg",
        "_index": "jobs",
        "_type": "couchbaseDocument",
        "_id": "217169",
        "_score": 0.8564255,
        "_source": { "meta": { "rev": "1-14435b27ed3300000000000000000000", "flags": 0, "expiration": 0, "id": "2171" } },
        "_explanation": {
          "value": 0.85642546,
          "description": "sum of:",
          "details": [
            {
              "value": 0.6953489,
              "description": "weight(doc.job_description:datas in 1043) [PerFieldSimilarity], result of:",
              "details": [ { "value": 0.6953489, "description": "score(doc=1043,freq=11.0), product of:", "details": [
                {
                  "value": 0.16107656,
                  "description": "weight(doc.job_description:administr in 1043) [PerFieldSimilarity], result of:",
                  "details": [ { "value": 0.16107656, "description": "score(doc=1043,freq=2.0), product of:", "details": [

```

Calcul du score par l'analyseur

Le champ description du résultat indique les résultats du classement et le calcul du poids tf-idf des termes database et administration.

Ces tests démontrent la bonne prise en compte de l'analyseur défini précédemment sur le champ job_description lors de l'indexation des données envoyées par Couchbase.

Nous avons vu que l'interface offerte par Elasticsearch offre de très bonne performances pour les requêtes de recherche avec une forte flexibilité. Pour autant cela ne remet pas en cause les services offerts nativement par Couchbase qui sont dédiés à des usages différents. C'est bien pour cette raison que le projet Couchbase propose un module d'intégration avec Elasticsearch. Couchbase s'intègre aussi avec Hadoop, Spark et Kafka. Concernant la partie recherche d'informations, il n'existe pas encore de module pour Solr, dommage, à l'issu du cours j'ai eu une préférence pour la simplicité et l'ergonomie de l'interface Solr comparé à celle d'Elasticsearch.

VI. Conclusion

Durant la réalisation de ce projet lié au module NFE204, j'ai pris conscience de la grande étendue des possibilités offertes par les systèmes documentaires distribués. Il existe en effet un grand nombre de systèmes aboutis permettant de gérer ces bases de données spécifiques. Ces mêmes systèmes ont été conçus pour s'intégrer avec d'autres offrants des services de recherche et d'analyse, des services de calcul distribués ou de passage à l'échelle. Dans le cadre du projet mené en autonomie, j'aurais souhaité pousser plus loin mes investigations car celui-ci a réellement éveillé ma curiosité. Il m'aurait fallu plus de temps disponible ou une implémentation concrète dans mon entreprise pour cela. Néanmoins, j'aurais appris qu'il est possible de trouver des ressources accessibles en Open Data, disponibles au grand public et que ces données peuvent être consommées par des systèmes basés sur le protocole http via les services REST. Qu'il est ensuite possible de transformer et de stocker ces données au sein de systèmes spécialisés dans les traitements documentaires. Que ces mêmes systèmes s'adaptent pour passer à l'échelle, répliquer leurs données et gérer les temps d'accès. Qu'enfin, les possibilités d'architectures en cluster sont multiples et que non seulement elles dépendent du besoin mais sont évolutives. Concrètement j'aurais ici récupéré des données mises à disposition par la ville de New-York, traité celles-ci par une application consommatrice et stockés les résultats dans la base de données documentaire Couchbase. J'aurais testé les différentes manières d'interroger la base, étudié les résultats quant aux performances en implémentant les vues et les fonctions de MapReduce, trouvé quelques applications directes : établissement de graphe et de cartes géographiques. Enfin j'aurais établi l'interface entre le système de gestion de la base de données documentaires et un serveur de recherche dédié avec un analyseur personnalisé. Même si au final l'application et l'interprétation des données a un intérêt limité, elle démontre néanmoins des possibilités offertes par ces systèmes.

VII. Annexes

1. Références

CONCEPTS COUCHBASE

- Les index

<http://developer.couchbase.com/documentation/server/4.1/concepts/indexing.html>

- Créer des documents dans Couchbase par l'api REST

<http://stackoverflow.com/questions/24528318/create-couchbase-documents-via-rest-api>

- Charger des fichiers JSON massivement dans Couchbase

<http://blog.couchbase.com/loading-json-data-couchbase>

- Le langage N1QL

<http://developer.couchbase.com/documentation/server/4.1/n1ql/n1ql-language-reference/index.html>

<http://docs.couchbase.com/files/Couchbase-N1QL-CheatSheet.pdf>

SOCRATA

- NYC Open Data

<https://nycopendata.socrata.com/>

- API SODA

<https://dev.socrata.com/>

- SoQL

<https://dev.socrata.com/docs/queries/>

CARTODB

<http://docs.cartodb.com/>

ELASTICSEARCH

- Mapping

<https://www.elastic.co/guide/en/elasticsearch/reference/current/mapping.html>

- Analyse

<https://www.elastic.co/guide/en/elasticsearch/reference/current/analysis.html>

2. Compléments

REQUETES N1QL AVEC CURL

- Extraire toutes les coordonnées GPS

```
curl -v http://localhost:8093/query/service -d "statement=SELECT unique_key, latitude, longitude FROM NYC"
> E:\outputGPS.txt
```

- Extraire les coordonnées GPS à la date du 31 décembre

```
curl -v http://localhost:8093/query/service -d "statement=SELECT unique_key, latitude, longitude, date FROM NYC WHERE STR_TO_MILLIS(date) BETWEEN STR_TO_MILLIS('2015-12-31') AND STR_TO_MILLIS('2016-12-31') "
> E:\outputGPS31Dec.txt
```

- Extraire les coordonnées GPS en Janvier sur Brooklyn

```
curl -v http://localhost:8093/query/service -d "statement=SELECT unique_key, latitude, longitude, date, borough FROM NYC WHERE borough = 'BROOKLYN' AND STR_TO_MILLIS(date) BETWEEN STR_TO_MILLIS('2015-01-01') AND STR_TO_MILLIS('2015-12-31') " > E:\outputGPSBrooklyn2016.txt
```

3. Interfaces

INTERFACE DE CREATION D'UNE VUE DANS COUCHBASE

NYC

> Views >

design/dev_AccidentQuartier/_view/AccidentQuartier

353632

Preview a Random Document

Edit Document

```
{
  "unique_key": "353632",
  "number_of_pedestrians_killed": "0",
  "borough": "BROOKLYN",
  "number_of_persons_killed": "0",
  "contributing_factor_vehicle_2": "Unspecified",
  "contributing_factor_vehicle_1": "Backing Unsafely",
  "location": {
    "latitude": "40.6469261",
    "needs_recoding": false,
    "longitude": "-74.0012504"
  },
  "latitude": "40.6469261",
  "number_of_pedestrians_injured": "0",
  "zip_code": "11232",
  "on_street_name": "42 STREET",
  "off_street_name": "7 AVENUE",
  "number_of_motorist_killed": "0",
  "date": "2014-06-18T00:00:00",
  "number_of_cyclist_killed": "0",
  "number_of_persons_injured": "0",
  "number_of_motorist_injured": "0",
  "longitude": "-74.0012504",
  "vehicle_type_code1": "SPORT UTILITY / STATION WAGON",
  "vehicle_type_code2": "PASSENGER VEHICLE",
  "number_of_cyclist_injured": "0",
  "time": "13:00"
}
```

```
{
  "id": "353632",
  "rev": "1-143b19b0bc3c0000000000000000000",
  "expiration": 0,
  "flags": 0
}
```

VIEW CODE

Save As...

Save

Map

```
1 function (doc, meta) {
2   emit(meta.id, null);
3 }
```

Reduce (built in: _count, _sum, _stats)

```
1
```

IMPLEMENTATION DU MAPREDUCE

294700

Preview a Random Document

Edit Document

VIEW CODE

Save As...

Save

Map

```
1 function (doc, meta) {
2   emit (1,doc.unique_key);
3 }
4
```

Reduce (built in: _count, _sum, _stats)

```
1 function (keys, values, rereduce) {
2
3   if (rereduce) {
4     return sum(values);
5   }
6   else {
7     return values.length;
8   }
9
10 }
11
```

Filter Results

?

7?full_set=true&inclusive_end=true&reduce=true&stale=false&connection_timeout=60000&limit=10&skip=0

Show Results

Development Time Subset

Full Cluster Data Set

Key	Value
null	763968
undefined	

4. Elasticsearch

MAPPING

```
22     "couchbaseDocument" : {
23       "_source" : {
24         "includes" : ["meta.*","doc.*"]
25       },
26       "properties" : {
27         "meta" : {
28           "type" : "object",
29           "include_in_all" : false
30         },
31         "doc": {
32           "dynamic": false,
33           "properties": {
34             "business_title": {
35               "type": "string",
36               "index": "not_analyzed"
37             },
38             "job_description": {
39               "type": "string",
40               "analyzer" : "JobAnalyzer"
41             }
42           }
43         }
44       }
45     }
```

ANALYSEUR

- EnglishAnalyser

[http://lucene.apache.org/core/4_7_2/analyzers-](http://lucene.apache.org/core/4_7_2/analyzers-common/index.html?org/apache/lucene/analysis/en/EnglishAnalyzer.html)

[common/index.html?org/apache/lucene/analysis/en/EnglishAnalyzer.html](http://lucene.apache.org/core/4_7_2/analyzers-common/index.html?org/apache/lucene/analysis/en/EnglishAnalyzer.html)